

PROBLEM STATEMENT THEME

Productivity Reimagined

- Employees spend too much time on repetitive work.
- Organizations lack visibility into productivity leakage.

TEAM NAME

Beyond the Workflow

TEAM MEMBER NAMES

Sunil Kumar, Ajmeera

Lakshmansamvith, Dunna

Prasanth, Varipireddi

PNC***NEXT***

IGNITING MINDS. A(G)CCELERATING IDEAS.

What problem are we solving?

Problem statement

Manual, repeated page-by-page regression checking is exactly the kind of hidden productivity drain the theme calls out. Teams know pages exist, but nobody can say which modules, sub-modules and actions automation actually exercises — so QA re-clicks the same pages every cycle, and that effort produces no report anyone can trust or share.

Key capabilities

Track module → sub-module → functionality coverage; surface untouched areas as gaps; run unattended via automation; publish one combined execution + coverage report.

Technology used

Playwright + TypeScript, a Manifest V3 browser extension, a CoverageEngine REST API, and a custom HTML/JSON test reporter.

Reusability

The page-object model and coverage layer are app-agnostic — any new module needs only a testdata/xpaths entry, no new framework code.

How does it work?

QA clicks through pages



CoverageEngine plugin



Automation scripts sync



Coverage + execution report

Solution flow / architecture

1 Browser launches with CoverageEngine extension loaded

2 Plugin session starts before test execution begins

3 Reusable Playwright scripts run acceptance-criteria tests, recording module / sub-module / functionality hits into that same session

4 Plugin session ends after execution completes

5 Reporter merges pass/fail results with the coverage gap analysis into one HTML + JSON report

What's different about our approach?

- Coverage is measured against a real module inventory, not guessed from test names
- The plugin session lifecycle is locked and fixed — test authors never touch capture logic
- XPaths and test data are separated from test logic, so new modules need no new framework code
- No manual clicking required once acceptance criteria are automated — the scripts sync with the plugin on their own

1 CoverageEngine plugin

A browser extension that watches regression testing as it happens — clicking through the application's pages — and shows which modules, sub-modules and actions were reached, and which were not.

Coverage model

Module — a route / page

Sub-module — a component

Functionality — an action / event

This three-level inventory is the "universe" every test-gap report is measured against.

Session lifecycle (locked)

- Extension loads into the browser on launch
- Session starts once, before any test runs
- Every action / route visit is captured into that session
- Session stops once, after all tests finish

This four-step contract is fixed on purpose, so capture behaves identically whether a person is clicking or automation is running.

2 Reusable automation scripts

Playwright test scripts built from finalised acceptance criteria. XPath's, test data and execution logic are kept apart, so the scripts sync with the CoverageEngine plugin automatically — no manual clicking required.

Separation of concerns

xpaths.properties — every locator, by key

testdata.properties — every expected value

BasePage — every click / fill / assert, logged

Page objects — which keys a page uses

Adding a module means adding data rows — never new framework code.

Coverage fixture

Every test automatically gets a **coverage** tracker via the shared test fixture — no per-test setup.

captureModule() — route reached

captureSubModule() — component exercised

captureFunctionality() — action / validation performed

Hits flush into the one active plugin session automatically.

Full execution logging

Every ACTION, VALIDATION, PASS, FAIL, SKIP and RETRY is written to:

the console, live

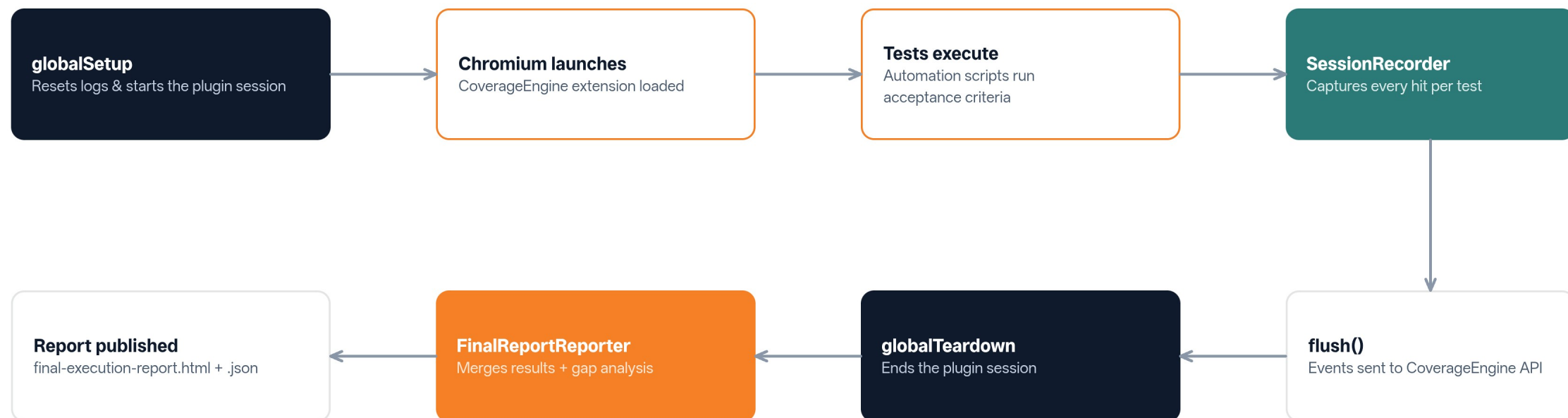
a per-test test-steps.log

one run-wide execution.log

The whole run can be replayed top-to-bottom from one file.

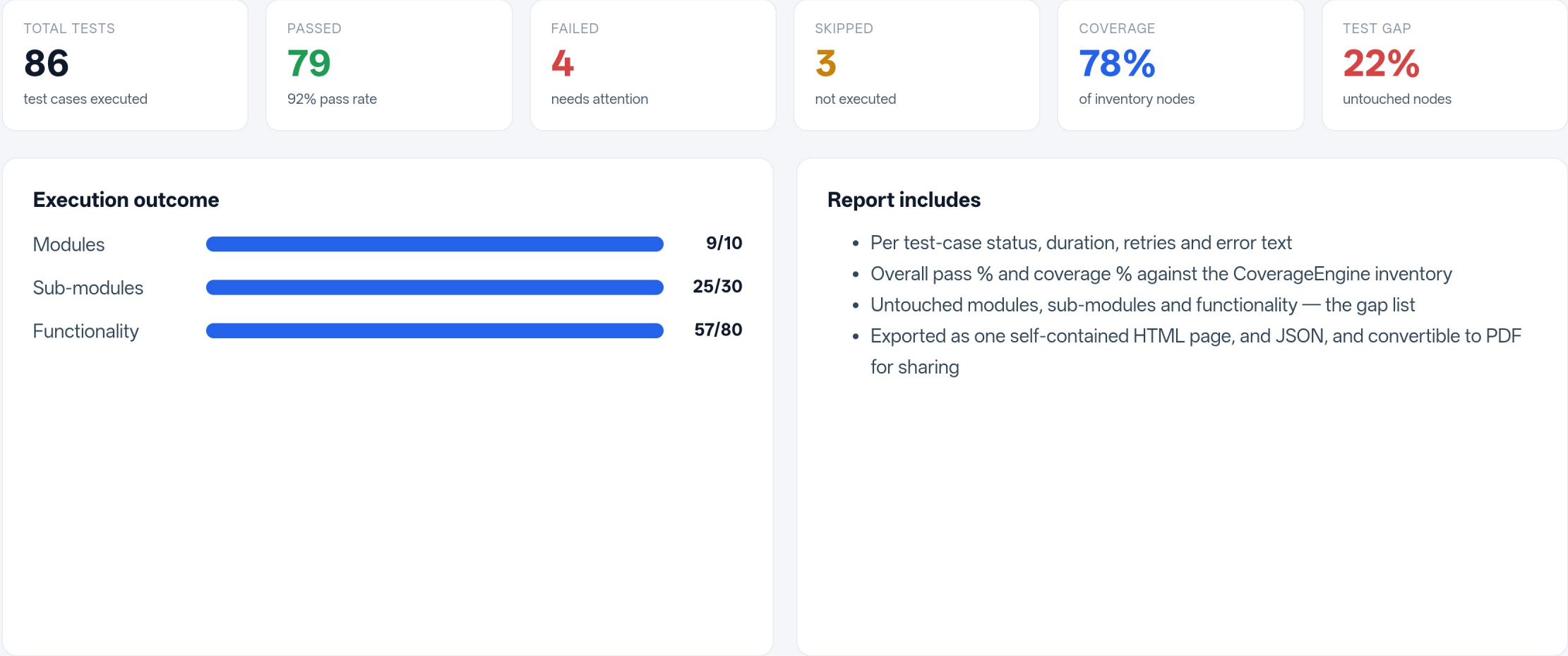
HOW THE TWO PARTS SYNC

One session, shared automatically



THE OUTPUT

One report: execution results and coverage gaps together



Business Benefits

BUSINESS IMPACT

Coverage gaps that used to be discovered in production are now visible before release. QA no longer has to manually click through every page to prove regression coverage — automation checks every page on every run and reports exactly what was and wasn't exercised, in a format leadership can open and trust.

Built & demonstrated

The full plugin session lifecycle, the reusable Playwright page-object framework, the coverage/gap analysis engine, and the combined HTML + JSON reporter are all implemented and working end to end.

Described, not yet built

Scheduled unattended runs, a PDF export step off the HTML report, and expanding the module inventory to cover every banking module.

QUANTIFY WITH WHICHEVER MEASURES APPLY

Time saved

Risk reduction

Quality improvement

Engineering velocity

Cost avoided

Every automated run now checks every page — and proves it

CoverageEngine plugin + reusable Playwright scripts, synced automatically, producing one shareable coverage and execution report.

PNC*NEXT*

IGNITING MINDS. A(G)CCELERATING IDEAS.

HACKATHON THEME

Problem Statement Theme

THEME

Productivity Reimagined

UNDERLYING BUSINESS PROBLEM

Organizations lack visibility into productivity leakage.

HACKATHON PROBLEM STATEMENT

Identify Hidden Productivity Drain

How might we uncover and eliminate the largest sources of productivity leakage across workflows and teams?

CoverageEngine targets this leakage directly: manual, repeated page-by-page regression checking